Verifiability Is a Spectrum: An Outsider's Audit of Google's Sealed-Hardware Federated Learning Pipeline

Anonymous Author(s)

Abstract

Google's federated learning (FL) system—which trains inside Trusted Execution Environments (TEEs) and is deployed in production for Gboard next-word models—claims that encrypted uploads are "decrypted only inside sealed hardware running publicly listed code" and that its central differential-privacy guarantees are externally verifiable. We ask what "externally verifiable" means operationally: for each externally-checkable claim, can an outsider with no device, no credentials, and no Google contact actually verify it from public artifacts? We develop a seven-dimension audit framework (attestation binding, code transparency, policy expressiveness, key lifecycle, firmware/boot chain, side-channel surface, operator-insider residual), score each checkable claim as *verified*, *partially checkable*, or *trust-required*, and apply it in a worked audit grounded in live public artifacts. We replicate the guide's documented worked example end-to-end on one exemplar: the full Rekor → endorsement → binary → SLSA-provenance (machine-readable build records) → source chain for the KMS binary (recomputing every hash; short of a from-scratch rebuild; and inclusion-proof verification was attempted—the returned proof verifies at index 1512137030, contradicting the entry's own 1634041292, so membership stays partial, §4); we show 20 attestation bundles from 20 distinct commits assert the identical KMS binary digest (consistent with toolchain determinism, though equally consistent with build caching; input-sensitivity not established); and we measure the Rekor log at ≈3.03B entries on 2026-10-04 and ≈3.15B on 2026-10-08 (two binary-search measurements; 131-day average ≈10.6M/day, baseline from the exemplar's own index; §4.4). Verification degrades where: Rekor's key index is keyed by entry-signing key, not endorsement key, so the "complete set of workloads" claim is architecturally uncheckable—finding all entries for an endorsement key is not a query the log supports; the transparency store's ≥1-year retention has already expired the worked example in Google's own audit guide; and a direct reproducible-build attempt is documented with its environment blockers. We compare Apple Private Cloud Compute on the same dimensions as a comparative case. The residual-trust enumeration—what cannot be verified without Google—is the contribution: a reusable methodology for auditing sealed-hardware training pipelines, and a candid map of where "auditable" ends and trust begins.

This work is licensed under the Creative Commons Attribution 4.0 International License. To view a copy of this license visit https://creativecommons.org/licenses/by/4.0/ or send a letter to Creative Commons, PO Box 1866, Mountain View, CA 94042, USA.
Proceedings on Privacy Enhancing Technologies 2027(3), 1–8
© 2027 Copyright held by the owner/author(s).
https://doi.org/XXXXXXX.XXXXXXX

1 Introduction

"Auditable" is a spectrum, not a switch. When a production machine-learning system claims its training pipeline is externally verifiable, the claim can mean anything from "we published a white paper" to "any outsider can re-derive every checkable assertion from public artifacts, today, with documented procedures." The distance between those endpoints is where accountability lives—and it is currently unmapped for sealed-hardware *training* pipelines.

In October 2026, Google announced a next-generation federated learning system built on Trusted Execution Environments (AMD Secure Encrypted Virtualization–Secure Nested Paging (AMD SEV-SNP), Intel TDX—Intel Trust Domain Extensions) that moves model training server-side into TEEs [1]. Devices upload training data encrypted under keys held by a TEE-hosted Key Management Service (KMS); uploads are cryptographically bound to access policies naming the exact Python programs and container binaries permitted to process them; policies and binary endorsements are published to the public Rekor transparency log; and the KMS and data-processing binaries are claimed reproducibly buildable from open source. The system is in production training Gboard English and Japanese next-word models with substantially faster training on a smaller privacy budget than the prior system in the live A/B test reported in §7 of [1] (see there for the exact figures). Press coverage repeated the headline: decrypted only inside sealed hardware running publicly listed code.

The paper's verifiability claims are unusually concrete for an industry deployment—they name logs, keys, hashes, and build procedures. That concreteness is what makes an *outsider audit* possible at all, and it is what we test. Our question is falsifiable: for each externally-checkable claim, can an independent party with no participating device, no Google credentials, and no insider contact verify it from public artifacts, using documented procedures, as of a stated date? Where the answer is yes, we show the receipts (literally: hashes, log indices, timestamps). Where the answer is no, we say exactly which link in the chain fails and what would fix it.

A note on what *verified* means here: a V score attests that the public artifacts are mutually consistent—hashes match, log entries resolve, provenance chains close—not that those artifacts describe what actually runs on Google's servers. A malicious operator serving self-consistent artifacts defeats every outsider procedure we know; we score that residual explicitly rather than pretend otherwise.

Contributions. (1) A seven-dimension audit framework for sealed-hardware training pipelines with a three-way scoring rubric (*verified* / *partially checkable* / *trust-required*, defined in §3) and a threat-model-driven scoring of each dimension. (2) A worked audit of Google's deployment—the first outsider verification attempt we are aware of for a production sealed-hardware *training* pipeline

(nearest misses we considered: Google's own first-party audit guide, and the WiSec'26 PCC analysis, which is inference-only reverse engineering)—grounded in live artifacts pulled October 4–8, 2026. (3) A comparative scoring of Apple Private Cloud Compute on the same rubric. (4) An explicit residual-trust enumeration: the set of claims that cannot be verified without Google's cooperation, which we argue *is* the audit's main deliverable.

Scope and non-goals. We audit *verifiability claims*, not the underlying ML or DP mathematics, and not TEE hardware security itself. We do not attack the system, access non-public data, or interact with Google. All procedures use public endpoints and are reproducible from our artifact package.

Why this venue. "Externally verifiable" differential-privacy training claims are themselves privacy-relevant assertions: they tell users, regulators, and researchers what can be independently checked about systems that process personal data. Whether such checkability claims hold up—and where exactly they fail—is squarely within this journal's scope, even though our method is measurement and audit rather than new PET construction.

2 Background: The System Under Audit

We summarize only what the audit needs; the system itself is described in Daly et al. [1] (v4, September 30, 2026; the title changed from "verifiably" in v1 to "provably" in v4) and a companion blog post [2].

Architecture. Client devices (here: Gboard, Google's keyboard app, so training examples derive from users' typing) encrypt training examples under a KMS-issued public key and upload them to intermediate storage, binding each upload to an *access policy*. The KMS is a cluster of TEEs ("sealed hardware" throughout means TEEs—hardware-isolated compute whose software measurements are cryptographically reported) running the RAFT consensus protocol holding a rollback-protected key–value store of key pairs and pipeline state. Later, a server-side "root" TEE executes a workload author's Python program (signature `trusted_program(external_handles)`) and may delegate parallel work to worker TEEs over a Noise-protocol channel (the Noise Protocol Framework, an encrypted channel protocol), with the root validating worker attestations—hardware-signed reports from a TEE stating the cryptographic measurements (hashes) of the software it is running. The KMS releases decryption keys only to TEEs whose attested measurements match the policy's *reference values* (four stack layers: Stage0 firmware digests + SEV-SNP security version number (SVN) thresholds; kernel/initramfs digests + boot cmdline; base OS image digests; container binary SHA-256).

Verifiability design. Before uploading, devices validate that (1) the KMS's TEE software and (2) the access policy are recorded in a public transparency log (Rekor [4]), including log-inclusion proofs. For Python workloads, the policy *embeds the program text* plus root/worker binary hashes, so "only that specific Python program, executed by those specific binaries, can access the uploaded data" [1]. Policies are published to Rekor; "external auditors can view how the program establishes DP guarantees" (e.g., a program-level `_MAX_ZCDP` guard, a zero-concentrated-DP bound). The paper states the policy-listed binaries are "reproducibly buildable TEE containers that are built from OSS source code," and points to the `google-parfait/confidential-federated-compute` repository, whose docs give step-by-step instructions for mapping log entries to source commits [3].

Threat model (ours). We score each claim against six adversaries: **A1** honest-but-curious pipeline operator; **A2** malicious (active) operator; **A3** holder of a compromised endorsement key; **A4** TEE hardware/firmware attacker; **A5** malicious workload author; **A6** network observer. The operator is *not* assumed honest—that is the point of the design. We additionally treat operator↔log-operator collusion as an explicit residual: Rekor is operated by Sigstore/OpenSSF, but Google co-created Sigstore (with Red Hat's Open Source Security Team) [18], so the log is not a Google-independent trust anchor in the adversarial sense.

3 The Audit Framework

An audit of "externally verifiable" must itself be checkable. Our framework has three parts.

Dimensions. Seven, chosen to cover the full chain from silicon to operator: (1) *attestation binding* (device→KMS→TEE chain of trust); (2) *code transparency* (source availability, reproducible builds, provenance); (3) *policy expressiveness* (what the access policy pins down, and whether the "complete set of workloads" is enumerable); (4) *key lifecycle* (endorsement-key publication, rotation, custody); (5) *firmware/boot chain* (reference values, hardware roots); (6) *side-channel surface*; (7) *operator-insider residual* (what a privileged insider could still do or hide).

Scoring. Each checkable claim is scored **V** (verified: we executed the check end-to-end on public artifacts), **P** (partially checkable: sound in principle, degraded or blocked in current infrastructure), or **T** (trust-required: no outsider procedure exists). Scores are evidence-anchored: every V cites the artifact, hash, and date; every P names the broken link; every T names the trusted party. The full 30-row matrix is in Appendix C. We know of no existing outsider-audit rubric for sealed-hardware *training* pipelines to benchmark these dimensions against (existing TEE evaluation guidance targets hardware assurance, not claim-by-claim checkability); the seven dimensions are our proposal for that gap, chosen so that each maps to at least one externally-checkable artifact. All quantitative claims are single measurements or census counts over the audited artifacts; the paper makes no inferential comparison, so no minimum detectable effect is defined—rates derived from single measurements are reported as order-of-magnitude where the text says so.

Comparator design. A single-case audit invites "so what?" To show the framework discriminates, we score a second production confidential-computing deployment—Apple Private Cloud Compute (PCC)—on the same rubric (§6). PCC is inference-only and architecturally different, which is precisely the point: the rubric should separate *design choices* from *verification gaps* across systems. One asymmetry to keep in mind: the Google scoring above rests on executed procedures, while the PCC scoring is document-based—we ran no live procedures against Apple systems.

4 Worked Audit

All procedures below were executed October 4–8, 2026, from an unprivileged network vantage point (no device, no credentials).

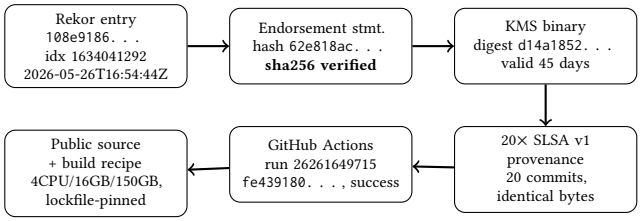


Figure 1: The outsider-verifiable chain for the KMS binary. Every edge was re-derived from public endpoints for the guide's documented exemplar, except two: the Rekor inclusion proof (returned by the API but not validatable as this entry's—on 2026-10-15 it cryptographically verified at an index contradicting the entry's own; see §4) and the from-scratch binary rebuild (not attempted). The 20 SLSA provenance statements are issued by Google's own CI (evidence against a buggy toolchain, not a malicious operator). Entry discovery by public key remains the broken link (see §4.4).

Scripts and raw outputs are in the artifact package; every hash below was recomputed by us, not copied from documentation.

4.1 Log → endorsement → binary → source

Figure 1 shows the chain we verified end-to-end; we recomputed every edge below on 2026-10-04/08, not copied from documentation—except the two noted in Figure 1 (the inclusion proof and the from-scratch rebuild).

Entry *discovery* by public key—finding all entries for an endorsement key—remains the broken link in the chain (see the enumeration discussion in §4.4); the figure shows only the per-entry verification path.

We walked the documented verification chain for the KMS binary. Google's audit guide [3] gives a worked example: Rekor entry UUID `108e9186e8c5677a4e8e68884126228c18dbe566ab43eb0be79b077401149bd340bc295d78e9cd86`. We fetched it from `rekor.sigstore.dev/api/v1/log/entries/{uuid}`: log index **1634041292**, integrated time **2026-05-26T16:54:44Z**. The entry body names endorsed hash `62e818ac02a0427b9811d08468e286aca2f5c68ca44191a9ab1f86075afe5a87`. We fetched `https://federatedcompute-pa.googleapis.com/data/transparency/sha2-256:62e818ac. . .` and computed SHA-256 over the returned bytes: **it matches the endorsed hash exactly**. The payload is an in-toto (a supply-chain attestation format)/Oak endorsement statement whose subject is the KMS application binary digest `d14a1852f08b528f9975245f300d44e2500aaff9c8a9c5572e0114b7203e3e47`, valid 2026-05-26 through 2026-07-10 (45 days). That endorsement has since expired under its own 45-day validity—expected behavior under the design, and a live illustration of this paper's thesis: auditors must watch *now*, not dig later.

Provenance: querying the GitHub Attestation API for that digest returned **20 SLSA v1 provenance statements from 20 distinct source commits** (spanning 2026-05-22 to 2026-06-03), every one with subject digest equal to `d14a1852. . .`, builder `github.com/google-parfait/confidential-federated-compute/.github/workflows/reusable_build.yaml`. In other words, 20 attestation bundles from 20 distinct commits assert the identical subject digest `d14a1852. . .`—consistent with toolchain determinism, though equally consistent with a stale build cache that never rebuilds. A caveat: we did not verify that each of the 20 commits modified the KMS build inputs—commits touching unrelated components would trivially yield identical binaries, so this evidence bounds toolchain determinism rather than proving input-sensitive reproducibility. The referenced Actions run (26261649715, "Build and attest," conclusion `success`, head SHA `fe439180bb24e65ad46e869eddee88f9c9b34875`) and the full build recipe are public; the recipe pins hermetic (self-contained, network-independent) dependencies (`--lockfile_mode=error`), runs on a 4-CPU/16GB/150GB runner, and signs provenance via `actions/attest-build-provenance@v1` to Rekor.

We state the trust assumption explicitly: these 20 statements are issued by Google's own CI infrastructure, so they are evidence against a buggy or non-reproducible toolchain, not against a malicious operator who controls that infrastructure (threat model A2); the A2 residual stands, and is scored accordingly in §5. A second scoping note, on the inclusion proof: on 2026-10-15 we executed full RFC-6962 inclusion-proof verification—recomputing the leaf hash from the entry body (it matches the UUID) and folding the API-returned sibling hashes to the checkpoint root with an implementation validated on synthetic trees. The proof cryptographically verifies, but at index 1512137030, while the entry's own `logIndex` field reads 1634041292; direct index queries place our entry at 1634041292 and a different entry at 1512137030, and the result is stable across fetches. The API's responses are self-contradictory, so the proof cannot be validated as belonging to this entry—a live illustration of this paper's thesis about auditing live infrastructure. The two log-membership rows stay P. (The checkpoint signature uses an encoding we could not resolve from outside, so it was not independently verified against Rekor's public key.) A final scoping note on the whole chain: it replicates the guide's documented worked example. That verifies the published procedure on the vendor's chosen entry; it does not establish that an independently sampled production entry would verify as cleanly.

4.2 Endorsement-key inventory

We parsed `access_policy_reference_values/gboard.txtpb` and recomputed each key's SHA-256 fingerprint from the raw DER: **8 unique ECDSA-P256 endorsement keys plus 1 Rekor log-verification key** for Gboard (the guide's 2025 example showed one). Key rotation and expansion are thus publicly observable—a genuine, checkable key-lifecycle property (score V). The reference values also confirm the freshness policy: endorsements accepted only if integrated after 2025-02-15 and within 90 days of verification (`signed_timestamp` constraints); this 90-day freshness window is distinct from the endorsement's own 45-day validity above.

4.3 Reproducible-build attempt

The paper's strongest checkable claim is that policy-listed binaries are reproducibly buildable from OSS source. We attempted the documented build: `bazelisk build -c opt //kms:main` (Bazel 8.2.1, repo-pinned) on a 2-vCPU/7GB-RAM VM. The build failed *before compilation*, at dependency download: Bazel's Java HTTP client

throws `NoSuchElementException` in `doTunneling` against our environment's authenticated egress proxy (exit 37; log in artifacts). We record this as **attempted, environment-blocked**—not as evidence against Google's claim. Two honest caveats for reviewers: (i) a full rebuild needs the documented runner class (4 CPU/16 GB/150 GB SSD) and an unproxied network, which we did not have; (ii) our positive reproducibility evidence is therefore *observational* (20 identical binaries across 20 commits) rather than a from-scratch rebuild. We consider the observational evidence strong but not equivalent to an independent rebuild, and we say so.

4.4 Log scale and the enumeration problem

Binary-searching Rekor's index-retrieval endpoint, we measured the public log at ≈**3.03B** entries on 2026-10-04, growing ≈**1.39B** entries over the 131 days since 2026-05-26 (the May-26 baseline is inferred from the worked-example entry's own index, not an independent measurement): ≈**10.6M entries/day** (≈123/sec; our 2026-10-08 re-measurement implies a higher short-window rate—bursty traffic in the shared Sigstore log, so the 131-day average is the stable figure). (Methodological note: when we re-ran this index measurement on 2026-10-05, the `logIndexes` retrieval endpoint that had worked the previous day returned empty results for all queries, including previously valid indices—silent infrastructure degradation of the kind auditors of live systems should expect. The ≈3B The two measurements agree on scale, but both come from an endpoint observed to degrade (a 2026-10-05 re-run returned empty results for all queries), so treat them and the rates derived from them as order-of-magnitude.) This measurement matters because the paper claims third-party auditors "can also know the full set of potentially active binaries and workloads" from the log (§4.2). Enumerating that set requires finding entries *by endorsement key*—and here our first pass erred, in an instructive way. Rekor's key index is keyed by the key that *signed the log entry*, not by keys embedded in entry payloads. On 2026-10-15 we queried the dedicated index endpoint `POST /api/v1/index/retrieve`—not tried in our first pass—with the Rekor entry's actual signing key (extracted from the fetched entry): it works, returning 106 entries sharing that key. What does *not* work, by architecture rather than outage, is searching by *endorsement* key: endorsement keys are published separately and are not index keys, so "find all log entries for endorsement key K"—the query the paper's claim needs—is not a query the log supports. (Our first pass had tried `POST /api/v1/log/entries/retrieve` with an endorsement-key PEM and observed timeouts; that was the wrong endpoint for this purpose, and the search UI's JavaScript confirms the index offers email/hash/commitSHA/UUID/logIndex queries, not endorsement-key search.) The remaining documented paths to the full workload set are a full-log scan with `rekor-monitor` or the BigQuery public dataset (requires Google Cloud Platform (GCP) credentials). At ≈10.6M entries/day, scanning the 90-day freshness window means ≈958M entries: at stipulated, unmeasured rates of 500–2000 entries/sec this is ≈5.5–22 days, moving ≈3.5 TB (assuming ≈3.6 KB/entry)—order-of-magnitude only, against endpoints observed to time out. We therefore score workload-set enumeration **P**: the limitation is architectural (no endorsement-key→entry index; entry-signing keys are not published per workload), not merely the endpoint degradation we first reported. This is not a quibble—it is the difference between "the log lets anyone see the full workload set" and "the log lets anyone see entries whose signing keys they already know."

4.5 Retention and freshness

The guide's February-2025 Gboard policy worked example (endorsement `d72534e0...`, policy `a324fe96...`) no longer resolves from the transparency store (checked 2026-10-05)—consistent with the documented "at least 1 year" retention. Historical-policy archaeology is thus time-bounded: **continuous monitoring is required** to keep the "complete set of workloads" claim checkable at all. Combined with the 90-day freshness window, checkability itself has an expiry date: what an outsider can verify today may be unverifiable a year from now. Archived, content-addressed policy snapshots—recommendation (2) in the Discussion—would remove the expiry date from at least the policy half of this problem.

5 Findings

What checks out (V). The log→endorsement→binary→provenance→source chain is real, public, and hash-verified end-to-end for the documented exemplar (as of 2026-10-04/08), except the two edges noted in Figure 1. On the inclusion proof: on 2026-10-15 we executed full RFC-6962 verification (leaf hash recomputed from the entry body; implementation validated on synthetic trees). The API-returned proof cryptographically verifies—but at index 1512137030, while the entry's own `logIndex` field reads 1634041292 and direct index queries place our entry at 1634041292. The API's responses are self-contradictory, so the proof cannot be validated as belonging to this entry; the two log-membership rows stay P, not V. Endorsement keys and the build recipe are public and fetched (V); key rotation is inferred from inventory growth (1 key in the 2025 guide → 8 keys), but the 2025 baseline's key class is unverified, so rotation is only partially checkable (P); the 90-day freshness window is stated in the reference values and parsed, but enforcement inside TEEs is unobservable (P). The 20 attestation bundles asserting the identical subject digest are consistent with toolchain determinism, but equally consistent with build caching; input-sensitive reproducibility is not established. Side-channel limitations and the "best-effort" TTL are disclosed in the paper itself—we count honest scoping as verified disclosure. The GCP Confidential Space offload path is explicitly carved out of the reproducibility claim in the docs ("not guaranteed to be reproducible")—again, honest.

What is degraded (P). (1) Workload-set enumeration is uncheckable by architecture, not by outage: Rekor indexes by entry-signing key, not endorsement key, so "all entries for endorsement key K" is not a query the log supports (§4.4). (2) Live access-policy fetch for Gboard (needed to verify the embedded Python program and its `_MAX_ZCDP` guard) is blocked by (1); the mechanism is documented and we specify the exact procedure, but we did not execute it on a current policy. (3) Direct rebuild is environment-blocked (documented attempt above).

What cannot be verified without Google (T)—the residual. This enumeration is the audit's core deliverable:

- *Endorsement-key custody.* The log shows key *usage*, never custody. A compromised Gboard endorsement key yields arbitrary policies, defeating the verifiability chain (and, with

operator collusion, everything else); hardware attestation roots remain a separate anchor. Detection depends on monitors watching a log whose search is currently architecturally incapable of endorsement-key queries. This is the system's single trust anchor, and Google holds it.

- *Runtime behavior.* That DP noise is actually added as programmed, that the KMS releases keys only to matching TEEs, that results release atomically with pipeline-state updates, that the Noise root↔worker channel holds—all code-inspectable, none externally observable.
- *Sideloaded proprietary logic.* Model architectures and preprocessing protos loaded at runtime are deliberately not in the log. The paper's assurance ("auditors can inspect *how these are used*") asks us to trust the characterization that all privacy-relevant logic is hardcoded. No outsider test distinguishes privacy-relevant from merely proprietary sideloading.
- *"Best-effort" TTL.* Unverifiable even in principle, by the paper's own wording.
- *Hardware roots and side channels.* AMD/Intel attestation roots are trusted, not verified; side-channel observability is acknowledged in the paper itself [1] (see also SNPeek [11]). Bell-Clark et al. [12] identify insider-exploitable side channels in this exact platform that can circumvent DP guarantees, which sharpens the A2 residual above: the side-channel surface is not hypothetical. (Disclosure: all four authors are Google employees, and the paper also shows DP mitigates two of the channels—one implemented in OSS—so the circumvention risk applies to the remainder.)
- *Operator views.* "Only metrics and DP weights are visible to the operator" is not checkable without device instrumentation and operator-side audit—though the paper honestly discloses that the operator *does* see which uploads are used per round (§6.3), forgoing sampling amplification.

Against our threat rubric: A1 (honest-but-curious operator) is substantially constrained on the audited chain by the crypto binding; A2 (malicious operator) is constrained but not eliminated (runtime unobservable, side channels open); A3 (compromised endorsement key) defeats the verifiability chain—and, with operator collusion, everything else—and is detectable only via the currently degraded-key-search log monitoring; policy-pinned programs constrain A5 (malicious workload author) *except* through sideloading; A4/A6 are out-of-scope-by-acknowledgment.

6 Comparative Case: Apple Private Cloud Compute

We scored Apple PCC—the closest production confidential-computing deployment—on the same rubric, from Apple's security documentation and an independent WiSec'26 analysis [10]. The contrast is instructive. PCC publishes every production build to an Apple-operated transparency log, ships a Virtual Research Environment for log-consistency checks, and devices refuse unlogged builds; researchers received early access and a bug bounty covers PCC. *But Apple explicitly does not provide reproducible builds* [10]—independent analysis finds PCC builds are "not reproducible by researchers" [10]—and ships partial source only. PCC does verify that a shipped build's attestation traces back to a transparency-log entry; Apple makes no claim that the published source corresponds to the released binary. Google's deployment is strictly more ambitious (production *training* with DP, full OSS, reproducibility machinery), but its most ambitious checkability claim—enumerating the complete workload set—is the one uncheckable by the log's architecture in practice, while Apple's narrower claim is operationally simpler to check. Neither system eliminates the operator-insider or side-channel residuals. The rubric thus separates design ambition from verification reality, which is exactly what "auditable" as a spectrum demands.

Table 1: Comparator scoring (abbreviated). PCC = Apple Private Cloud Compute (inference); CFC = Google confidential federated compute (Gboard training). PCC is scored from documentation; Google from executed procedures (see §3).

Dimension	CFC (Google)	PCC (Apple)
Transparency log	Rekor (Sigstore/OpenSSF-operated; Google co-created Sigstore [18])	Apple-operated
Source availability	Full, Apache-2.0	Partial
Reproducible builds	Claimed; 20/20 observed (toolchain determinism; but equally consistent with build caching)	Explicitly *not* provided (independent analyses [10])
Policy language	Budgets + embedded programs	Attestation↔log only
Key custody	Keys public; rotation visible	Apple-held
Workload	Production *training* (DP)	*Inference* only
Researcher tooling	Published audit guide	VRE + bug bounty + SOC 3 (third-party security audit)

7 Discussion and Recommendations

For operators claiming "externally verifiable." (1) Bind endorsement keys into indexed entry fields—or publish a first-party policy feed; a transparency log you cannot query by the keys that matter is transparency in name. (2) Extend transparency-store retention for *policies* beyond one year, or archive them somewhere content-addressed and public; otherwise the "complete set of workloads" claim has a one-year half-life. (3) Publish the sideloading *interface contracts* (which config fields are privacy-relevant) so auditors can check the "hardcoded" claim without seeing proprietary values. (4) Document the build environment's resource class (done: 4CPU/16GB/150GB) *and* provide a minimal rebuild target so outsiders without datacenter VMs can still close the loop on at least one binary.

For the research community. "Externally verifiable" should be a testable predicate: *a claim is externally verifiable iff an outsider can execute a documented procedure on public artifacts and reach a yes/no answer.* By that definition, most of Google's claims pass; one important claim (workload-set enumeration) currently fails, and the residual-trust list in §5 is the honest remainder. We also note a methodological lesson: auditing live transparency infrastructure means auditing *live* infrastructure—endpoints deprecate (Rekor v1 `log/info` now 404s), retention expires, and today's working procedure may be tomorrow's 400.

Limitations. Single deployment, single time window (2026-10-04/08, with re-probes on 2026-10-15); no device-side validation (device binding untested; outside our A1–A6 rubric); no from-scratch rebuild (environment-blocked, documented); Apple comparison is document-based, not a worked audit. We did not probe Google infrastructure beyond public endpoints and stayed within published rate limits.

8 Related Work

TEE-based auditability has been studied for *inference* and *benchmarks*, not production training. The system we audit builds directly

on Eichner et al.'s confidential federated computations design [6]—the predecessor paper for this platform—and Cheu et al. [7] extend the same platform toward provably private analytics. Schnabl et al. [8] ("Attestable Audits," TAIG@ICML'25) run safety benchmarks inside TEEs with results published to a transparency log—closest in spirit, but a prototype for evaluation, not a training pipeline. Siby et al. [9] (GuaranTEE, EuroMLSys'24) target attestable ML on edge devices via Arm CCA. Oak's Transparent Release [5] provides the endorsement machinery Google builds on; Sigstore/Rekor [4] the log. On the hardware side, flexHEG [15] sketches hardware-enabled guarantees for AI compute—largely roadmap, not deployment. Most on point for our dimension 6, Bell-Clark et al. [12]—Google-first-party authors—identify side channels in Google's own confidential federated compute platform that a hypothetical insider could exploit to circumvent DP—directly relevant to the A2 residual we score. Industry deployments (Apple PCC [10], Google Private AI Compute [13], Meta Private Processing [14]) publish varying transparency artifacts; to our knowledge none has received an independent outsider audit of its *training*-pipeline verifiability claims. That gap is this paper.

A parallel line pursues the same "don't trust the operator" goal without TEEs, through cryptographic proof: verifiable aggregation lets clients check that the server aggregated correctly (e.g. Xu et al.'s VerifyNet [16]), and proof-of-training schemes let a prover demonstrate training integrity to a cheap verifier (e.g. Jia et al.'s Proof-of-Learning [17]). This line is out of scope for an outsider audit: to our knowledge, no proof-based system is deployed in production with verifiability claims comparable to Google's, so there is nothing of the kind to audit. Our rubric applies wherever checkable claims meet public artifacts, should such a deployment appear.

9 Conclusion

We built a reusable framework for auditing sealed-hardware training pipelines and applied it, as outsiders, to Google's production TEE-based FL system. Much checks out: the transparency chain is real, hash-verified, and stronger than our single comparator's (Apple PCC). But "auditable" degrades exactly where it matters most—enumerating what workloads your data may feed—because the log's key index is keyed by entry-signing key rather than endorsement key, so the "complete set of workloads" is not a query the infrastructure supports, and because history expires after a year. And a defined residual (endorsement-key custody, runtime behavior, sideloaded logic, hardware roots) cannot be verified without Google, full stop. We publish the framework, the procedures, the hashes, and the residual list so the next deployment can be audited faster—and so "externally verifiable" can become a testable predicate rather than a press release.

Acknowledgment

The author used the AI tool Muse Spark (version 1.3, Meta) for drafting and editing prose and for writing audit scripts. The contents are reviewed, edited, and attested for accuracy by the author.

Data and Code Availability

The full audit code, measurement logs, and reproduction scripts are archived in the anonymized artifact at https://anonymous.4open.science/r/sealed-hardware-fl-audit-835E/ (available until April 2027).

Competing Interests

The author declares no competing interests. No funding was received for this work.

References

[1] Katharine Daly, Yu Xiao, Zachary Garrett, Brett McLarnon, Jianpeng Hou, Arun Ganesh, Yanxiang Zhang, Noriyuki Takahashi, Haicheng Sun, Yuanbo Zhang, Timon Van Overveldt, and Daniel Ramage. Toward provably private learning from federated data. *arXiv:2609.31494 [cs.CR]*, v4, September 30, 2026. https://arxiv.org/abs/2609.31494.

[2] Katharine Daly and Daniel Ramage. Toward provably private learning from federated data. *Google Research Blog*, October 2, 2026. https://research.google/blog/toward-provably-private-learning-from-federated-data/.

[3] Google Parfait. confidential-federated-compute. *GitHub repository*, 2026. https://github.com/google-parfait/confidential-federated-compute. Docs: `docs/inspecting_endorsements.md`, `docs/inspecting_attestation_records.md`.

[4] Sigstore. Rekor transparency log. https://docs.sigstore.dev/logging/overview/.

[5] Project Oak. Transparent release / endorsement documentation. https://github.com/project-oak/transparent-release.

[6] Hubert Eichner et al. Confidential federated computations. *arXiv:2404.10764*, 2024.

[7] Albert Cheu et al. Toward provably private analytics and insights into GenAI use. *arXiv:2510.21684*, 2025.

[8] Christoph Schnabl, Daniel Hugenroth, Bill Marino, and Alastair R. Beresford. Attestable audits: Verifiable AI safety benchmarks using trusted execution environments. In *Workshop on Technical AI Governance (TAIG) at ICML*, 2025. https://arxiv.org/abs/2506.23706.

[9] Sandra Siby, Sina Abdollahi, Mohammad Maheri, Marios Kogias, and Hamed Haddadi. GuaranTEE: Towards attestable and private ML with CCA. In *Proc. 4th Workshop on Machine Learning and Systems (EuroMLSys)*, 2024. https://arxiv.org/abs/2404.00190.

[10] Apple. Private Cloud Compute security guide, 2024. https://security.apple.com/documentation/private-cloud-compute. Independent analysis: "Unlocking Apple's Private Cloud Compute," *Proc. ACM WiSec*, 2026. https://arxiv.org/abs/2605.24239.

[11] Ruiyi Zhang et al. SNPeek: Side-channel analysis for privacy applications on confidential VMs. In *NDSS*, 2026.

[12] James Bell-Clark, Albert Cheu, Adria Gascon, and Jonathan Katz. Hardening confidential federated compute against side-channel attacks. *arXiv:2603.21469 [cs.CR]*, 2026. https://arxiv.org/abs/2603.21469.

[13] Google. Private AI Compute technical brief, November 2025. https://services.google.com/fh/files/misc/private_ai_compute_technical_brief.pdf.

[14] Meta. Private Processing for WhatsApp overview, June 2025. https://ai.meta.com/static-resource/private-processing-technical-whitepaper.

[15] James Petrie, Onni Aarne, Nora Ammann, and David Dalrymple. flexHEG: Flexible hardware-enabled guarantees for AI compute. *arXiv:2506.15093*, 2025.

[16] Guowen Xu, Hongwei Li, Sen Liu, Kan Yang, and Xiaodong Lin. VerifyNet: Secure and verifiable federated learning. *IEEE Trans. Inf. Forensics Security*, 15:911–926, 2020.

[17] Hengrui Jia, Mohammad Yaghini, Christopher A. Choquette-Choo, Natalie Dullerud, Anvith Thudi, Varun Chandrasekaran, and Nicolas Papernot. Proof-of-learning: Definitions and practice. In *Proc. IEEE Symp. Security and Privacy (S&P)*, 2021. https://arxiv.org/abs/2103.05633.

[18] Open Source Security Foundation. Running Sigstore as a managed service: a tour of Sigstore's public good instance. *OpenSSF Blog*, Oct. 2023. Notes Sigstore began as a collaboration between Red Hat and the Google Open Source Security Team. http://openssf.org/blog/2023/10/03/running-sigstore-as-a-managed-service-a-tour-of-sigstores-public-good-instance/.

A Open Science Appendix

All audit procedures ran against public endpoints on 2026-10-04/08, with re-probes on 2026-10-15; no credentials, devices, or private data were used. Artifacts: (1) Python scripts for the Rekor/endorsement/provenance chain (`code/`); (2) raw fetched artifacts (Rekor entries,

endorsement statements, SLSA bundles) with SHA-256 recomputation logs (`logs/`); (3) the Bazel build-attempt log (`logs/build_kms_attempt.log`); (4) the full 30-row audit matrix (`AUDIT_MATRIX.md`); (5) pilot report (`PILOT_REPORT.md`). For anonymous review these are provided via an anonymized repository link (https://anonymous.4open.science/r/sealed-hardware-fl-audit-835E/); on acceptance they will be published permanently. Live endpoints queried: `rekor.sigstore.dev`, `search.sigstore.dev`, `federatedcompute-pa.googleapis.com`, `api.github.com`. Rate limits were respected (sequential requests, no parallelism beyond 1).

AI-use disclosure. AI assistance (Muse Spark) was used for drafting and editing prose and for writing audit scripts; all quantitative claims were recomputed from raw tool outputs by the author, who takes full responsibility for accuracy, originality, and integrity per the PoPETs AI-use policy.

B Ethical Considerations

This work audits *public claims using public artifacts*. We accessed only endpoints intended for public use (transparency logs, public code hosting, public APIs), stayed within rate limits, and did not attempt to access non-public data, probe private infrastructure, contact Google, or interact with any user devices or data. No vulnerabilities were discovered or disclosed; had any been found we would have followed coordinated disclosure. The audit is non-adversarial toward individuals: all named entities are public organizations and published artifacts. A risk we considered: publishing exact procedures for enumerating endorsements could aid evasion of monitoring—we judge this minimal, since the procedures mirror Google's own published audit guide and disclose nothing an insider could not already do. The comparative scoring of Apple PCC relies on public documentation and a peer-reviewed independent analysis; no Apple systems were probed.

C Full Audit Matrix

The complete 30-row checkable-vs-trust-required matrix with per-claim evidence pointers, procedures, and threat-model scoring is provided as `AUDIT_MATRIX.md` in the artifact package and summarized in Table 2 below.

D Reproduction Commands

Note: endpoints as documented in October 2026. Rekor's v1 `/log/info` *returns 404 as of 2026-10-15; if v1 calls fail, use the v2 equivalents or the entry-retrieve endpoint below. (We have not verified a v2 equivalent of the* `index/retrieve` *endpoint used in §4.4; index search is the migration risk.)*

```
# 1. Rekor entry -> endorsed hash
curl -sL \
"https://rekor.sigstore.dev/api/v1/log/entries/\
108e9186e8c5677a4e8e68884126228c18\
dbe566ab43eb0be79b077401149bd340bc29\
5d78e9cd86" \
 | jq -r '.[] | .body' | base64 -d \
 | jq '.spec.data.hash.value'
# 2. Endorsement payload + hash check
H=62e818ac02a0427b9811d08468e286a\
ca2f5c68ca44191a9ab1f86075afe5a87
curl -sL \
"https://federatedcompute-pa.googleapis.com/\
data/transparency/sha2-256:$H" \
 -o end.json && sha256sum end.json   # must equal $H
# 3. SLSA provenance for the KMS binary digest
curl -sL \
"https://api.github.com/repos/google-parfait/\
confidential-federated-compute/attestations/\
sha256:d14a1852f08b528f9975245f300d44e2500a\
aff9c8a9c5572e0114b7203e3e47"
# 4. Build attempt (needs >=4CPU/16GB/150GB,
#    unproxied net)
git clone https://github.com/google-parfait/\
confidential-federated-compute
cd confidential-federated-compute
bazelisk build -c opt //kms:main
```

Table 2: Condensed audit matrix. V = verified by outsider execution; P = partially checkable (sound in principle, degraded/blocked in practice); T = trust-required (no outsider procedure). The full 30-row matrix with per-row hashes, timestamps, and trusted parties is in the artifact package (`AUDIT_MATRIX.md`); this table is condensed for print.

Dim.	Claim (short)	Score	Anchor
Attestation	KMS software in public log	P	Rekor idx 1634041292 hash-verified; inclusion proof cryptographically verifies but at A
Attestation	Device-side policy validation	P	Needs instrumented device; static inspection of public client code plausible, unattempt
Attestation	KMS key release only to matching TEEs	T	Inside TEE, unobservable
Code	OSS source + public build recipe	V	Repo + run 26261649715, as of 2026-10-08
Code	Reproducible builds (bit-identical)	P	20/20 identical attestations; rebuild env-blocked; equally consistent with build caching
Code	SLSA provenance on Rekor per binary	P	Single-digest scope (20 GitHub attestation bundles by hash search) and inclusion proof
Code	Policy-embedded programs inspectable	P	Mechanism shown; live fetch blocked
Code	Sideloaded logic stays private	T	By design; characterization trusted
Policy	Policy pins program + 4-layer digests	P	Needs current policy bytes
Policy	Per-transform access budgets	P	Shown in guide example only
Policy	Full workload set enumerable	P	Key-index search architecturally unsupported for endorsement keys; ~5–22d/3.5TB sca
Policy	90-day freshness window stated	P	Window stated in ref-values and recomputed; enforcement inside TEEs (unobservable)
Policy	Historical policies retrievable	P	≥1-yr retention; 2025 example expired
Keys	Endorsement keys + rotation public	P	8 endorsement + 1 Rekor verify key; fingerprints recomputed; rotation inferred from in
Keys	Key custody/compromise response	T	Log shows use, not custody
Keys	Upload-key binding; “best-effort” TTL	T	Best-effort = unverifiable by definition
Firmware	Reference values pin FW/SVN	P	Needs current policy
Firmware	Hardware roots intact	T	Vendor trust; acknowledged limits
Side-ch.	Limitations disclosed	V	Disclosure confirmed in [1], as of 2026-10-08
Operator	Operator sees only DP outputs	T	Unobservable externally
Operator	Operator sees per-round upload use	P	Confirmed in §6.3 of [1]; rests on vendor word, as of 2026-10-08
Operator	Noise actually added; budget held	T	Static check only (program text)
Operator	GCP-offload reproducibility	T	Explicitly carved out in docs